

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 18-09-2026

1. TITLE

Working with Node.js Built-in (Core) Modules and User-Defined Modules

2. AIM

To explore Node.js's built-in core modules — os, path, fs, url, and events — and to create and use a custom user-defined module, in order to understand Node.js's module system based on require() and module.exports.

3. OBJECTIVE

The objectives of this experiment are:

- To import and use Node.js core modules without installing any external packages.
- To retrieve system information using the os module.
- To work with file and directory paths using the path module.
- To perform synchronous file operations using the fs module.
- To parse a URL into its components using the url module.
- To create and respond to custom events using the events module.
- To create a user-defined module, export a function from it using module.exports, and import it into another file using require().

4. THEORY

Node.js follows the CommonJS module system, where every .js file is treated as a separate module with its own scope. Node.js ships with a set of built-in "core" modules that provide access to operating-system, file-system, and networking functionality without needing npm install. The os module exposes information about the underlying operating system, such as platform() (the OS platform), arch() (CPU architecture), and homedir() (the user's home directory). The path module provides utilities for working with file and directory paths in a platform-independent way, such as basename() (the last portion of a path) and extname() (a file's extension). The fs (File System) module allows reading and writing files; writeFileSync() performs a synchronous, blocking write, meaning the program waits for the write to finish before continuing. The url module's URL class parses a URL string into structured properties such as host and pathname. The events module exports an EventEmitter class that implements the observer pattern: on() registers a listener for a named event, and emit() triggers that event, invoking every listener registered for it. In addition to these core modules, developers can create their own modules: any value assigned to module.exports in a file becomes what other files receive when they call require() with that file's relative path, which is how the custom getStudent function is shared between files in this experiment.

5. ALGORITHM / PROCEDURE

1. Create a file predefined_modules.js and import the built-in os, path, fs, url, and events modules using require().
2. Use os.platform(), os.arch(), and os.homedir() to print the operating system, CPU architecture, and home directory to the console.

3. Use `path.basename(__filename)` and `path.extname(__filename)` to print the current file's name and extension.
4. Use `fs.writeFileSync()` to synchronously create a file named `test.txt` containing the text "Hello Node.js", and log a success message.
5. Construct a new `url.URL` object from a sample URL string and print its host and pathname properties.
6. Create a new `EventEmitter` instance, register a listener for a custom "welcome" event using `on()`, and trigger it using `emit()`.
7. Run `predefined_modules.js` using the command "node predefined_modules.js" and observe the console output.
8. Create a second file `student.js` that defines a `getStudent(name, rollNo)` function returning a formatted string, and export it using `module.exports`.
9. Create a third file `usedefined.js` that imports the exported function from `student.js` using `require("./student")` and calls it with sample values.
10. Run `usedefined.js` using the command "node usedefined.js" and observe the console output.

6. CODE EXPLANATION

`predefined_modules.js` demonstrates five of Node.js's built-in core modules in one file. It first imports `os`, `path`, `fs`, `url`, and the `EventEmitter` class from `events`, all without any external installation. It then calls `os.platform()`, `os.arch()`, and `os.homedir()` to print basic system information, and calls `path.basename(__filename)` and `path.extname(__filename)` to inspect the currently running file's own name and extension using Node's special `__filename` variable. Next, `fs.writeFileSync("test.txt", "Hello Node.js")` synchronously creates (or overwrites) a text file in the working directory before logging a confirmation message. A new URL object is then constructed from a sample link, and its host and pathname properties are printed to show how the `url` module breaks a URL into structured parts. Finally, an `EventEmitter` instance registers a "welcome" listener with `on()` and immediately fires it with `emit("welcome")`, printing a greeting. `student.js` is a small user-defined module: it declares a `getStudent(name, rollNo)` function that builds and returns a formatted string, and exposes that single function to other files by assigning it to `module.exports`. `usedefined.js` then imports that function with `const student = require("./student")` — the `./` prefix tells Node.js to resolve `student.js` relative to the current file rather than searching `node_modules` — and calls `student("Rama karri", 87)`, logging the formatted string that the function returns.

7. PROGRAM

```
predefined_modules.js
const os = require("os");
const path = require("path");
const fs = require("fs");
const url = require("url");
const EventEmitter = require("events");

// OS module
console.log("OS:", os.platform());
console.log("CPU:", os.arch());
console.log("Home Directory:", os.homedir());

// Path module
console.log("File Name:", path.basename(__filename));
```

```

console.log("Extension:", path.extname(__filename));

// File System module
fs.writeFileSync("test.txt", "Hello Node.js");
console.log("File created successfully");

// URL module
const myURL = new url.URL("https://example.com:8080/page?id=10");
console.log("Host:", myURL.host);
console.log("Path:", myURL.pathname);

// Events module
const event = new EventEmitter();

event.on("welcome", () => {
    console.log("Welcome to Node.js!");
});

event.emit("welcome");
student.js
function getStudent(name, rollNo) {
    return "Name: " + name + ", Roll No: " + rollNo;
}

module.exports = getStudent;
usedefined.js
const student = require("./student");

console.log(student("Rama karri", 87));

```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run "node predefined_modules.js"	OS platform, CPU architecture, and home directory are printed	System details printed correctly (e.g. "OS: linux")	Pass
TC-02	Check path module output	File name and extension of the running script are printed	Printed "File Name: predefined_modules.js" and "Extension: .js"	Pass
TC-03	Check fs module output	test.txt is created with the text "Hello Node.js" and a success message is logged	File created and "File created successfully" printed	Pass
TC-04	Check url module	Host and	Printed "Host:"	Pass

	output on a sample link	pathname of the URL are printed separately	example.com:8080" and "Path: /page"	
TC-05	Check events module output	Emitting the "welcome" event triggers its listener	Printed "Welcome to Node.js!"	Pass
TC-06	Run "node usedefined.js"	The getStudent function from student.js returns a formatted string with the given name and roll number	Printed "Name: Rama karri, Roll No: 87"	Pass

9. OUTPUT

Console output of "node predefined_modules.js":

```

$ node predefined_modules.js
OS: linux
CPU: x64
Home Directory: /home/rama
File Name: predefined_modules.js
Extension: .js
File created successfully
Host: example.com:8080
Path: /page
Welcome to Node.js!

```

Console output of "node usedefined.js":

```

$ node usedefined.js
Name: Rama karri, Roll No: 87

```

10. RESULT

Thus, Node.js's built-in os, path, fs, url, and events modules were successfully explored, and a user-defined module (student.js) was created and imported into another file using module.exports and require(). Both scripts were executed successfully and produced the expected console output.